

R7 — PWM: o pino, a carga e o limite do módulo

Período, razão cíclica e resolução medidos, e o que a ventoinha e o buzzer fazem com eles

Microcontroladores — DENE/UFMT

OBJETIVOS

- Medir período, frequência e razão cíclica do PWM, e confrontá-los com PR2 e o divisor do Timer2.
- Distinguir o sinal no pino do microcontrolador do sinal na carga, através do driver.
- Medir a razão cíclica de partida e de parada da ventoinha pela rotação, contada com o Timer1.
- Trocar frequência por resolução, e ouvir o resultado.
- Reconhecer os limites do módulo: os 100% que não cabem e a frequência mínima.

CH2-1 (LCD)	ON — mostra a razão cíclica, a configuração e a contagem	NESTE KIT
CH5-3 e CH5-4	OFF, obrigatoriamente	
Chaves SWITCHS (PORTB)	OFF	
CH3-5 (COOLER)	ON — ventoinha em RC2, a saída do CCP1	
CH3-2 (SPEED)	ON — tacômetro em RC0, contado pelo Timer1 como no R6	
CH3-6 (BUZZER)	OFF — só entra na Parte 5, no lugar do cooler	
CH3-3, CH3-4 e CH3-7	OFF	
Botões	INT0 (SW12, RB0) sobe 10%, INT1 (SW13, RB1) desce 10%, INT2 (SW14, RB2) troca a configuração	

Com CH3-2 em ON, **não pressione o botão de RC0**: ele aterra o tacômetro. Os registradores estão na **folha de referência dos temporizadores**.

PERIGO

O ponto de teste COOLER chega a 12 V. Mantenha a garra de terra de **todas** as pontas de prova no mesmo terra do kit, e nunca a prenda no ponto de teste.

Pontuação. As seis tarefas somam 8,0, e as previsões P1 a P5 da aula 6, preenchidas antes da sessão, valem 2,0. A nota de cada tarefa e de cada previsão está na margem, ao lado dela. A seção *Se sobrar tempo*, no fim, não vale nota: é para quem terminar antes.

Osciloscópio: ponta de prova em 10× e compensada, **acoplamento DC**, filtro de largura de banda desligado, disparo na borda de subida do canal 1. Em acoplamento AC a onda quadrada vira uma rampa.

BANCADA

1 Previsões — entregues no início da sessão

As mesmas perguntas do fim da aula 6. Quem trouxe a folha preenchida copia aqui as respostas; a nota é do raciocínio, não do número.

P1. Com PR2 = 255 e divisor 16, qual frequência você espera medir no pino do cooler?

PREVISÃO · 0,4 pt

P2. Variando a razão cíclica de 0 a 100% em passos de 10%, em qual passo o rotor começa a girar? Justifique pelo torque de partida.

PREVISÃO · 0,4 pt

P3. Depois de partir, reduzindo a razão cíclica, o rotor para no mesmo valor em que partiu?

PREVISÃO · **0,4 pt**

P4. Com 50%, o que se ouve no cooler a 977 Hz? E a 20 kHz?

PREVISÃO · **0,4 pt**

P5. Pedindo 100% com PR2 = 255, o que aparece no osciloscópio? E com PR2

PREVISÃO · **0,4 pt**

2 199?

3 O programa

Um programa só, para a sessão inteira. Ele gera o PWM no CCP1 (RC2), conta o tacômetro no Timer1 durante janelas de 1 s do Timer0 — como no R6 — e mostra tudo no display. INT2 percorre quatro configurações do Timer2:

C	PR2	Divisor	Frequência	Resolução
0	255	1:16		
1	255	1:4		
2	255	1:1		
3	199	1:1		

Complete as duas últimas colunas antes de gravar: é a conta da aula 6.

```
#define _XTAL_FREQ 16000000UL
#include <xc.h>
#include <stdint.h>
#include "lcd.h"

#define JANELA 49911u /* 1 s: 65536 - 15625, divisor 1:256 (R6) */

static const uint8_t pr2_cfg[4] = { 255, 255, 255, 199 };
static const uint8_t t2con_cfg[4] = { 0x06, 0x05, 0x04, 0x04 }; /* 1:16, 1:4, 1:1, 1:1 */

static void pwm_iniciar(void) /* na ordem da folha de dados (aula 6) */
{
    PR2 = pr2_cfg[0]; /* 1. periodo */
    CCP1L = 0; /* 2. razao inicial: 0 % */
    LATCbits.LATC2 = 0; /* 3. pino como saida */
    TRISCbits.TRISC2 = 0;
    T2CON = t2con_cfg[0]; /* 4. Timer2 ligado */
    CCP1CON = 0x0C; /* 5. modo PWM, saida simples */
}

static void pwm_razao(uint16_t razao) /* 0 a 1023 */
{
    if (razao > 1023u) {
        razao = 1023u;
    }
    CCP1L = (uint8_t)(razao >> 2);
}
```

```

    CCP1CON = (uint8_t)((CCP1CON & 0xCF) | (uint8_t)((razao & 0x03u) << 4));
}

/* passo de 0 a 10 -> razao em passos do periodo atual: 4 * (PR2 + 1) */
static uint16_t razao_do_passo(uint8_t passo, uint8_t c)
{
    return (uint16_t)((uint32_t)passo * 4u * ((uint16_t)pr2_cfg[c] + 1u) / 10u);
}

static void mostrar(uint8_t passo, uint8_t c, uint16_t pulsos)
{
    lcd_posicao(0, 0);
    lcd_texto("D=");  lcd_numero((uint16_t)passo * 10u, 3);
    lcd_texto("% C="); lcd_numero(c, 1);
    lcd_posicao(1, 0);
    lcd_texto("N=");  lcd_numero(pulsos, 5);  lcd_texto(" /s");
}

void main(void)
{
    uint8_t passo = 5, c = 0;
    uint8_t a0 = 1, a1 = 1, a2 = 1; /* estado anterior dos botoes */
    uint16_t pulsos = 0;

    ADCON1 = 0x0F; /* RB0-RB2 e RE0-RE1 digitais */
    TRISB |= 0x07; /* botoes INT0, INT1, INT2 */
    TRISCbits.TRISC0 = 1; /* RC0 = T13CKI: tacometro */

    lcd_iniciar();
    pwm_iniciar();
    pwm_razao(razao_do_passo(passo, c));

    TMR1H = 0; TMR1L = 0;
    T1CON = 0x83; /* 16 bits, 1:1, sincronizado, pino RC0 */
    TMR0H = (uint8_t)(JANELA >> 8); TMR0L = (uint8_t)(JANELA & 0xFF);
    T0CON = 0x87; /* 16 bits, interno, 1:256 */
    INTCONbits.TMR0IF = 0;

    for (;;) {
        uint8_t b;

        b = PORTBbits.RB0; /* INT0: sobe 10 % */
        if (b != a0) {
            if (b == 0u && passo < 10u) { passo++; pwm_razao(razao_do_passo(passo, c)); }
            a0 = b; __delay_ms(20); /* espera o repique passar */
        }
        b = PORTBbits.RB1; /* INT1: desce 10 % */
        if (b != a1) {
            if (b == 0u && passo > 0u) { passo--; pwm_razao(razao_do_passo(passo, c)); }
            a1 = b; __delay_ms(20);
        }
        b = PORTBbits.RB2; /* INT2: proxima configuracao do Timer2 */
        if (b != a2) {
            if (b == 0u) {
                c = (uint8_t)((c + 1u) % 4u);
                PR2 = pr2_cfg[c];
                T2CON = t2con_cfg[c];
                pwm_razao(razao_do_passo(passo, c));
            }
        }
    }
}

```

```

    }
    a2 = b; __delay_ms(20);
}

if (INTCONbits.TMR0IF) {      /* fechou a janela de 1 s */
    INTCONbits.TMR0IF = 0;
    TMR0H = (uint8_t)(JANELA >> 8);  TMR0L = (uint8_t)(JANELA & 0xFF);

    uint8_t lo = TMR1L;
    pulsos = ((uint16_t)TMR1H << 8) | lo;
    TMR1H = 0;  TMR1L = 0;

    mostrar(passo, c, pulsos);
}
}
}

```

razao_do_passo escala a razão pelo período da configuração atual, e por isso “50%” continua sendo 50% quando PR2 muda. Com PR2 = 255, o passo 10 pede 1024, e pwm_razao corta em 1023 — é a Tarefa 5.

NOTA

4 Parte 1 — o PWM no pino

Tarefa 1. Configuração C=0. Meça RC2 no conector de PORTC e varie a razão cíclica com INT0 e INT1.

TAREFA · 2,0 pt

Frequência prevista em P1: _____ Medida: _____

Razão nominal	t_{alto} medido	Período medido	Razão medida
20%			
50%			
80%			

Agora fixe 50% e percorra as configurações com INT2:

C	Frequência medida	Razão medida
0		
1		
2		
3		

(a) O período mudou quando você mudou a razão cíclica? E a razão, quando mudou a configuração?

(b) A frequência medida confere com a tabela? Se divergir, o erro está em PR2, no divisor, ou na hipótese sobre F_{osc} ? Um desvio de exatamente 25% nas quatro linhas aponta para qual deles?

(c) De C=0 a C=2 a frequência subiu dezesseis vezes sem perder resolução. De C=2 a C=3 ela subiu só 28%, e perdeu resolução. Por quê?

O núcleo roda a 16 MHz, e não a 48 MHz. Os bits de configuração gravados com o bootloader dão 48 MHz ao periférico USB e 16 MHz à CPU; o `#pragma config` da aplicação é ignorado. Isso foi descoberto por medição, não por leitura: uma onda programada para 300 ms mediu 375 ms, e o fator de 1,25 revelou o relógio verdadeiro. A alínea (b) é o mesmo raciocínio.

DIVERGÊNCIA

5 Parte 2 — o pino contra a carga

Tarefa 2. C=0, razão 50%. Canal 1 em RC2; canal 2 no ponto de teste COOLER, depois do driver ULN2803. Antes de ligar o canal 2: as duas ondas terão a mesma amplitude? A mesma fase?

TAREFA · 1,0 pt

Desenhe as duas formas de onda, uma sobre a outra, com as escalas anotadas.

	RC2	COOLER
Nível baixo		
Nível alto		

(a) As ondas estão em fase ou invertidas? Explique pelo funcionamento do driver.

(b) Aparece algum pico na borda em que a saída sobe? De onde ele vem?

“Pino em nível alto” e “carga energizada” são afirmações diferentes, e a diferença está no driver. Todo driver de potência faz alguma transformação desse tipo — inversão, deslocamento de nível, atraso — e conhecer a do seu é parte do projeto, não detalhe de montagem.

CONCEITO

6 Parte 3 — a partida e a parada do rotor

Tarefa 3. C=0. Parta de 0% e suba de 10 em 10% com INT0, esperando o N estabilizar em cada passo. Depois desça de 100% a 0% com INT1.

TAREFA · 2,0 pt

Razão	Subindo: gira?	Subindo: N	Descendo: gira?	Descendo: N
0%				
10%				
20%				
30%				
40%				
50%				
60%				

70%

80%

90%

100%

(a) Em que razão o rotor partiu? Em que razão parou? Confere com P2 e P3? Explique a diferença.

(b) A rotação é proporcional à razão cíclica? Use a coluna N.

O tacômetro de uma ventoinha de três fios é alimentado pela mesma linha que o PWM chaveia. Durante o tempo desligado ele pode ficar sem alimentação, e o sinal em SPEED pode ganhar pulsos falsos na frequência do PWM — ou perder os verdadeiros. **Olhe SPEED no osciloscópio em 50%** antes de confiar no N das razões intermediárias. Se o sinal estiver picotado, registre isso na tabela: a contagem certa de um sinal errado continua errada.

DIVERGÊNCIA

7 Parte 4 — frequência contra resolução, de ouvido

Tarefa 4. Razão em 50%. Aproxime o ouvido do cooler e percorra as quatro configurações.

TAREFA · 1,0 pt

C	O que se ouve	N (rotação)
0 — 977 Hz		
1 — 3,9 kHz		
2 — 15,6 kHz		
3 — 20 kHz		

(a) Confere com a P4? De onde vem o som, se a rotação não muda?

(b) Qual configuração você escolheria para a ventoinha do termostato, e o que ela custa?

8 Parte 5 — os limites do módulo

Tarefa 5. Leve a razão a 100% (D=100). Em C=2 e depois em C=3, amplie a base de tempo do osciloscópio até ver o início de cada período.

TAREFA · 1,0 pt

C	O que aparece no início do período
2 (PR2 = 255)	
3 (PR2 = 199)	

Confere com a P5? Por que um e não o outro?

Tarefa 6. Desligue CH3-5 e ligue CH3-6 (BUZZER). Sem regravar nada, ouça as quatro configurações com 50%, e depois varie a razão cíclica em $C=0$.

TAREFA · 1,0 pt

(a) O que muda no som quando se troca a configuração? E quando se troca a razão?

(b) O lá de 440 Hz tocado no O1 é possível com o módulo? E o de 880 Hz? Se algum for, dê PR2 e o divisor. Qual é o primeiro lá possível?

O módulo de PWM foi feito para controlar potência, onde frequências de quilohertz são o normal. Para notas graves, ele não serve, e a saída volta a ser o software — que alcança qualquer frequência, mas ocupa o processador. A interrupção do encontro 7 resolve isso.

CONCEITO

9 Armadilhas frequentes

Sintoma	Causa provável
Onda com bordas arredondadas	Ponta em 1× ou não compensada
Onda quadrada vira rampa	Acoplamento AC no canal
Nenhum sinal em RC2	TRISC2 em entrada, Timer2 desligado, ou CCP1CON fora de 0x0C
A razão cíclica não muda	Escreveu em CCPR1H, que é só leitura no modo PWM
Os botões não respondem	ADCON1 não é 0x0F: RB0–RB2 são AN12, AN10 e AN8 (R5)
Display com lixo	ADCON1 deixa RE0/RE1 analógicos (AN5 e AN6)
N sempre zero	CH3-2 em OFF, TRISC0 em saída, ou TMR1CS = 0

10 Entrega

As tabelas das Tarefas 1, 3, 4 e 5; o desenho da Tarefa 2 com escalas.

TAREFA

Responda também:

(a) No R6, o Timer2 não apareceu; aqui ele não conta nada que o programa leia. O que o processador faz para manter a onda no pino?

(b) Tempo, eventos, forma de onda: qual temporizador fez cada coisa entre o R6 e o R7, e qual deles dependeu do laço?

11 Se sobrar tempo

E1 — a rampa sem espera. Troque o controle por botões pela rampa da aula 6: a cada estouro de 10 ms do Timer0 (pré-carga 0x63C0), a razão sobe ou desce 4 unidades. Com o buzzer desligado e o cooler ligado, observe a rotação acompanhando a rampa.

OPCIONAL

E2 — a contagem limpa. Em 50%, compare o N com a frequência de SPEED no osciloscópio. Se o tacômetro estiver picotado pelo PWM, a contagem de 100% continua valendo como referência? Por quê?

OPCIONAL

E3 — o segundo canal. O CCP2 usa o mesmo Timer2. Configure CCP2CON = 0x0C e uma razão diferente em CCPR2L, e meça RC1 com as chaves CH3-3 e CH3-4 em OFF — o CCP2 só sai em RC1 se o CCP2MX do bootloader o puser lá. As duas saídas têm a mesma frequência? A mesma razão?

OPCIONAL

No R8: a interrupção, por inteiro — o repique que o R6 revelou, tratado de verdade, e o lá de 440 Hz que o módulo não alcança, tocado por um temporizador enquanto o display é atualizado.

NOTA